

SIKSHA 'O' ANUSANDHAN
DEEMED TO BE UNIVERSITY

Admission Batch:

Session:

Laboratory Record

**Python for Computer Science and Data Science 2 (CSE
3651)**

Submitted by

Name: _____

Registration No.: _____

Branch: _____

Semester: _____ Section: _____



**Department of Computer Science & Engineering Faculty of
Engineering & Technology (ITER)**

Jagamohan Nagar, Jagamara, Bhubaneswar, Odisha - 751030

INDEX

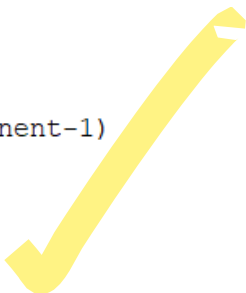
[illegible]

MINOR ASSIGNMENT-2: COMPUTER SCIENCE THINKING: RECURSION, SEARCHING, SORTING AND BIG O

1. Write a recursive function `power(base, exponent)` that, when called, returns $\text{base}^{\text{exponent}}$
Example: `power(3,4) = 3*3*3*3`. Both base and exponent are user-defined and assume that the exponent is an integer greater than or equal to 1.

Ans.

```
#Name: Arman Singh
#RegdNo-2241019468
def power(base,exponent):
    if exponent == 0:
        return 1
    else:
        return base * power(base,exponent-1)
base = int(input("Enter base: "))
pove = int (input("Enter Power: "))
result = power(base,powe)
print("Result: ",result)
```



Output:

```
Enter base: 2
Enter Power: 3
Result: 8
```

2. The greatest common divisor of integers x and y is the largest integer that evenly divides into both x and y . Write and test a recursive function `gcd` that returns the greatest common divisor of x and y . The `gcd` of x and y is defined recursively as follows: If y is equal to 0, then `gcd(x,y)` is x ; otherwise, `gcd(x,y)` is `gcd(y,x%y)`.

Ans.

```
#Name: Arman Singh
#RegdNo-2241019468
def gcd(x, y):
    if y == 0:
        return x
    else:
        return gcd(y, x%y)
print(gcd(48, 18)) # Output: 6
```

3. Write a recursive function that takes a number n as an input parameter and prints n -digit strictly increasing numbers.

Ans.

```
#Name: Arman Singh
#RegdNo-2241019468
def generate_numbers(n, start=1, num=""):
    if n == 0:
        print(num)
        return
    for i in range(start, 10):
        generate_numbers(n - 1, i + 1, num + str(i))

n = int(input("Enter the number of digits: "))
generate_numbers(n)
```

Output

```
Enter the number of digits: 2
12
13
14
15
16
17
18
19
```

4. Implement a recursive solution for computing the nth Fibonacci number. Then, analyze its time complexity. Propose a more efficient solution and compare the two approaches.

Ans.

#Name: Arman Singh
#RegdNo-2241019468

```
def fib(n):
    if (n==0 or n ==1):
        return n
    return fib(n-1)+fib(n-2)
print(fib(10))
#timecomplexity o(n)
#tabulation method
def fib2(n):
    if n == 0:
        return 0
    elif n == 1:
        return 1
    dp=[0]*(n+1)
    dp[0]=0
    dp[1]=1
    for i in range(2,n+1):
        dp[i]=dp[i-1]+dp[i-2]
    return dp[n]
print(fib2(10))
```

```
----- RESTART: D:\DOO.
55
55
|
```

First method results in exponential time complexity : $O(2^n)$

Second method result in linear time complexity: $O(n)$

Q5. Given an array of N elements, not necessarily in ascending order, devised an algorithm to find the kth largest one. It should run in $O(N)$ time on random inputs.

Ans.

#Name: Arman Singh

#RegdNo-2241019468

```
def kthlargest(arr, k):  
    return quickselect(arr, 0, len(arr)-1, len(arr)-k)  
def quickselect(arr, left, right, k):  
    if left==right:  
        return arr[left]  
  
    pivotind = partition(arr, left, right)  
    #sorted array  
  
    if pivotind ==k:  
        return arr[pivotind]  
    elif pivotind>k:  
        return quickselect(arr, left, pivotind-1, k)  
    else:  
        return quickselect(arr, pivotind+1, right, k)  
def partition(arr, left, right):  
    pivot=arr[right]  
    i=left  
    for j in range(left, right):  
        if arr[j]<=pivot:  
            arr[i], arr[j]=arr[j], arr[i] #swap  
            i+=1  
    arr[i], arr[right]=arr[right], arr[i]  
    return i
```

```
arr=[13,4,5,2,10]  
print("arr[13,4,5,2,10]")  
num = int(input("Enter k value: "))  
larg=kthlargest(arr, num)  
print("Largest: ", larg)
```

Output:

```
arr[13,4,5,2,10]  
Enter k value: 2  
Largest: 10  
|
```

6. For each of the following code snippets, determine the time complexity in terms of Big O. Explain your answer.

(a)

```
1 def example1(n):  
2     for i in range(n):  
3         for j in range(n):  
4             print(i, j)
```

(b)

```
1 for i in range(n):  
2     print(i)
```

(c)

```
1 def recursive_function(n):  
2     if n <= 1:  
3         return 1  
4     return recursive_function(n - 1) + recursive_function(n - 1)
```

A. Time Complexity: $O(n^2)$

I. The outer loop runs n times.

II. The inner loop runs n times for each iteration of the outer loop.

III. Total iterations: $n \times n = n^2$.

IV. Big O Complexity: $O(n^2)$ (Quadratic time).

B.

Time Complexity: $O(n)$

i. The loop runs n times.

ii. Each iteration performs a constant-time operation ($O(1)$).

iii. Big O Complexity: $O(n)$ (Linear time).

C.

i. Time Complexity: $O(2^n)$

ii. This is a tree recursion, where each call spawns two recursive calls. The number of recursive calls forms a binary tree of depth n

iii. The total number of calls is approximately 2^n in the worst case.

iv. Big O Complexity: $O(2^n)$ (Exponential time).

Q7. Write Given N points on a circle, centered at the origin, design an algorithm that determines whether there are two points that are antipodal, i.e., the line connecting the two points goes through the origin. Your algorithm should run in time proportional to $N \log N$.

```
#Name: Arman Singh
#RegdNo-2241019468
import math
def has_ap(points):
    angles = [math.atan2(y,x) for x,y in points]
    #sort angles
    angles.sort()

    #check if any 2points are exactly pi radians apart
    n=len(angles)
    for i in range(n):
        for j in range(i+1,n):
            if(math.isclose(abs(angles[j]-angles[i]),math.pi,rel_tol=1e-9)):
                return True #found antipodal points
    return False

#eg
points = [(1, 0), (-1, 0), (0, 1), (0, -1), (math.sqrt(2)/2, math.sqrt(2)/2), (-:
print(has_ap(points)) # Output: True
```

8. Output: The quicksort algorithm is a recursive sorting technique that follows these steps:

a) Partition Step: Choose the first element of the array as the pivot and determine its final position in the sorted array by ensuring all elements to its left are smaller and all elements to its right are larger.

b) Recursive Step: Recursively repeat the partitioning process on the subarrays created on either side of the pivot. As an example, consider the array [37, 2, 6, 4, 89, 8, 10, 12, 68, 45] with 37 as the pivot. Using the partitioning logic, the pivot eventually moves to its correct position, resulting in two subarrays: [12, 2, 6, 4, 10, 8] and [89, 68, 45]. The algorithm continues recursively until the entire array is sorted. Write a Python function quick sort that implements the quicksort algorithm. The function should include a helper function quick sort helper to handle recursion. The helper function must take a starting and ending index as arguments and sort the array in-place. Demonstrate the function by sorting the given array and printing the sorted output.

```
#Name: Arman Singh
#RegdNo-2241019468
def quick_sort(arr):
    """Main function to call quick_sort_helper for sorting the array."""
    quick_sort_helper(arr, 0, len(arr) - 1)

def quick_sort_helper(arr, start, end):
    """Helper function that sorts the array in-place using QuickSort."""
    if start < end:
        pivot_index = partition(arr, start, end)
        quick_sort_helper(arr, start, pivot_index - 1) # Sort left subarray
        quick_sort_helper(arr, pivot_index + 1, end)   # Sort right subarray

def partition(arr, start, end):
    """Partitions the array using the first element as the pivot."""
    pivot = arr[start]
    low = start + 1
    high = end

    while True:
        while low <= high and arr[low] <= pivot: # Move low forward if elemer
            low += 1
        while low <= high and arr[high] > pivot: # Move high backward if elen
            high -= 1
        if low <= high:
            arr[low], arr[high] = arr[high], arr[low] # Swap misplaced elemer
        else:
            break

    arr[start], arr[high] = arr[high], arr[start] # Swap pivot with arr[high]
    return high # Return pivot index

# Example array
arr = [37, 2, 6, 4, 89, 8, 10, 12, 68, 45]
quick_sort(arr)
print("Sorted array:", arr)
```

Output:_____

```
Sorted array: [2, 4, 6, 8, 10, 12, 37, 45, 68, 89]
```

9. You are given the following list of famous personalities with their net worth:

- Elon Musk: \$433.9 Billion
- Jeff Bezos: \$239.4 Billion
- Mark Zuckerberg: \$211.8 Billion
- Larry Ellison: \$204.6 Billion
- Bernard Arnault & Family: \$181.3 Billion
- Larry Page: \$161.4 Billion

Develop a program to sort the aforementioned details on the basis of net worth using

- a. Selection sort
- b. Bubble sort
- c. Insertion sort.

The final sorted data should be the same for all cases. After you obtain the sorted data, present the result in the form of the following dictionary:

```
{'name1':networth1, 'name2':networth2,...}
```

Ans.

```
#Name: Arman Singh
#RegdNo-2241019468
personalities = [("Elon Musk", 433.9), ("Jeff Bezos", 239.4),
                 ("Mark Zuckerberg", 211.8), ("Larry Ellison", 204.6),
                 ("Bernard Arnault & Family", 181.3), ("Larry Page", 161.4)]

def selection_sort(data):
    arr = data[:] # Copy the list to avoid modifying original
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i + 1, n):
            if arr[j][1] < arr[min_idx][1]: # Compare net worth
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr

def bubble_sort(data):
    arr = data[:]
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j][1] > arr[j + 1][1]: # Swap if out of order
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr

def insertion_sort(data):
    arr = data[:]
    n = len(arr)
    for i in range(1, n):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j][1] > key[1]:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr

# Sorting using all three methods
sorted_selection = selection_sort(personalities)
sorted_bubble = bubble_sort(personalities)
sorted_insertion = insertion_sort(personalities)
# Convert sorted list to dictionary
sorted_dict = {name: net_worth for name, net_worth in sorted_selection}
# Print sorted dictionary
print("Sorted Dictionary:", sorted_dict)
```


Output:

```
Sorted Dictionary: {'Larry Page': 161.4, 'Bernard Arnault & Family': 181.3, 'Larry Ellison': 204.6, 'Mark Zuckerberg': 211.8, 'Jeff Bezos': 239.4, 'Elon Musk': 433.9}
```

Q10. Use Merge Sort to sort a list of strings alphabetically. Example: Input: ['apple', 'orange', 'banana', 'grape'] Output: ['apple', 'banana', 'grape', 'orange']

Ans.

```
#Name: Arman Singh
#RegdNo-2241019468
def merge_sort(arr):
    if len(arr) <= 1:
        return arr # Base case: already sorted

    mid = len(arr) // 2 # Find the middle index
    left_half = merge_sort(arr[:mid]) # Recursively sort left half
    right_half = merge_sort(arr[mid:]) # Recursively sort right half
    return merge(left_half, right_half)

# Merge sorted halves
def merge(left, right):
    sorted_list = []
    i = j = 0
    # Merge elements in order
    while i < len(left) and j < len(right):
        if left[i] < right[j]: # Compare strings alphabetically
            sorted_list.append(left[i])
            i += 1
        else:
            sorted_list.append(right[j])
            j += 1

    # Add remaining elements
    sorted_list.extend(left[i:])
    sorted_list.extend(right[j:])

    return sorted_list

# Example Input
words = ["apple", "orange", "banana", "grape"]
sorted_words = merge_sort(words)
# Output the sorted list
print("Sorted List:", sorted_words)
```

Output

```
Sorted List: ['apple', 'banana', 'grape', 'orange']
```

Q11. Without using the built-in `sorted()` function, write a Python program to merge two pre-sorted lists into a single sorted list using the logic of Merge Sort. Example: Input: [1, 3, 5, 7] and [2, 4, 6, 8] Output: [1, 2, 3, 4, 5, 6, 7, 8]

Ans.

#Name: Arman Singh

#RegdNo-2241019468

```
def merge_sorted_lists(list1, list2):  
    merged = []  
    i = j = 0  
  
    while i < len(list1) and j < len(list2):  
        if list1[i] < list2[j]:  
            merged.append(list1[i])  
            i += 1  
        else:  
            merged.append(list2[j])  
            j += 1  
  
    merged.extend(list1[i:])  
    merged.extend(list2[j:])  
    return merged
```

```
list1 = [1, 3, 5, 7]  
list2 = [2, 4, 6, 8]  
result = merge_sorted_lists(list1, list2)  
print(result)
```

Output

```
[1, 2, 3, 4, 5, 6, 7, 8]
```